```
import pandas as pd

df = pd.read_csv('/content/news.csv')
display(df.head())
```

	Unnamed: 0	title	text	label
0	8476	You Can Smell Hillary's Fear	Daniel Greenfield, a Shillman Journalism Fello...	FAKE
1	10294	Watch The Exact Moment Paul Ryan Committed Pol...	Google Pinterest Digg Linkedin Reddit Stumbleu...	FAKE
2	3608	Kerry to go to Paris in gesture of sympathy	U.S. Secretary of State John F. Kerry said Mon...	REAL
3	10142	Bernie supporters on Twitter erupt in anger ag...	— Kaydee King (@KaydeeKing) November 9, 2016 T...	FAKE
4	875	The Battle of New York: Why This Primary Matters	It's primary day in New York and front-runners...	REAL

```
import numpy as np
import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import string

# Download NLTK data if not already downloaded
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
print(f"Dataset size (rows, columns): {df.shape}")

# Identify input text and target labels
X = df['text']
y = df['label']

print("\nClass distribution of target labels:")
display(y.value_counts())
```

```
Dataset size (rows, columns): (6335, 4)

Class distribution of target labels:
```

label	count
REAL	3171
FAKE	3164

dtype: int64

```
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()

    # Remove punctuation
    text = ''.join([char for char in text if char not in string.punctuation])

    # Tokenize text
    tokens = word_tokenize(text)

    # Remove stopwords
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]

    # Join tokens back into a string
    return ' '.join(tokens)

# Apply the preprocessing function to the 'text' column
```

```
df['cleaned_text'] = df['text'].apply(preprocess_text)

print("DataFrame with 'cleaned_text' column:")
display(df[['text', 'cleaned_text']].head())
```

```
DataFrame with 'cleaned_text' column:
```

	text	cleaned_text
0	Daniel Greenfield, a Shillman Journalism Fello...	daniel greenfield shillman journalism fellow f...
1	Google Pinterest Digg Linkedin Reddit Stumbleu...	google pinterest digg linkedin reddit stumbleu...
2	U.S. Secretary of State John F. Kerry said Mon...	us secretary state john f kerry said monday st...
3	— Kaydee King (@KaydeeKing) November 9, 2016 T...	— kaydee king kaydeeking november 9 2016 lesso...
4	It's primary day in New York and front-runners...	primary day new york frontrunners hillary clin...

```
# Initialize TfidfVectorizer
tfidf_vectorizer = TfidfVectorizer(max_features=5000) # Limiting to 5000 features for demonstration

# Fit and transform the cleaned text data
X_vectorized = tfidf_vectorizer.fit_transform(df['cleaned_text'])

print(f"Feature matrix shape: {X_vectorized.shape}")

# Display sample feature names (vocabulary)
sample_feature_names = tfidf_vectorizer.get_feature_names_out()[:20] # Get first 20 feature names
print("\nSample feature names:")
print(sample_feature_names)
```

```
Feature matrix shape: (6335, 5000)

Sample feature names:
['10' '100' '1000' '10000' '100000' '11' '12' '1237' '13' '14' '15' '150'
 '16' '17' '18' '19' '1948' '1960s' '1968' '1970s']
```

```
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_vectorized, y, test_size=0.2, random_state=42)

print(f"Training set features shape: {X_train.shape}")
print(f"Testing set features shape: {X_test.shape}")
print(f"Training set labels shape: {y_train.shape}")
print(f"Testing set labels shape: {y_test.shape}")
```

```
Training set features shape: (5068, 5000)
Testing set features shape: (1267, 5000)
Training set labels shape: (5068,)
Testing set labels shape: (1267,)
```

```
from sklearn.naive_bayes import MultinomialNB

# Initialize the Multinomial Naive Bayes model
naive_bayes_model = MultinomialNB()

# Train the model on the training data
naive_bayes_model.fit(X_train, y_train)

print("Multinomial Naive Bayes model trained successfully.")
print("\nModel parameters:")
display(naive_bayes_model.get_params())
```

```
Multinomial Naive Bayes model trained successfully.

Model parameters:
{'alpha': 1.0, 'class_prior': None, 'fit_prior': True, 'force_alpha': True}
```

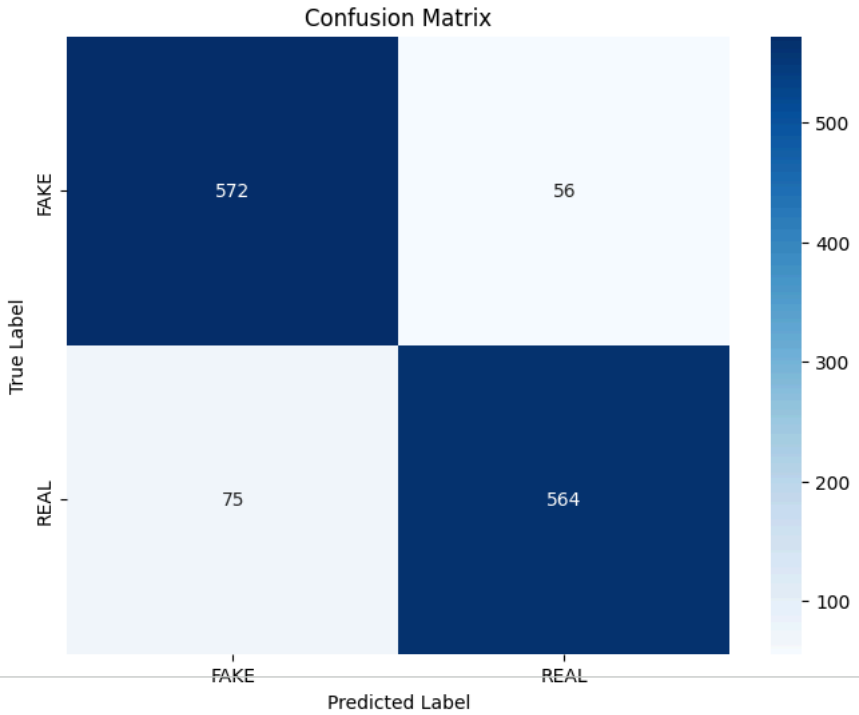
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# Make predictions on the test set
y_pred = naive_bayes_model.predict(X_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
```

```
# Generate Classification Report (Precision, Recall, F1-score)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Generate Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(conf_matrix)

# Visualize Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=naive_bayes_model.classes_, yticklabels=naive_bayes_model.classes_)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

```
Accuracy: 0.8966

Classification Report:
              precision    recall  f1-score   support

        FAKE       0.88      0.91      0.90       628
        REAL       0.91      0.88      0.90       639

    accuracy                           0.90      1267
   macro avg       0.90      0.90      0.90      1267
weighted avg       0.90      0.90      0.90      1267


Confusion Matrix:
[[572  56]
 [ 75 564]]
```